

LLM (Large Language Model):

Model trained on huge amount of text data (Wikipedia, Books, Web text etc) helps it to understand and generate human like language.

LLM Providers:

Provider	Models
OpenAI	GPT3, GPT3.5, GPT-4, GPT-4o, GPT-5 etc.
Anthropic	Claude 1, 2, 3
Meta	LLaMA 2, LLaMA 3
Google	Gemini

etc.....

GPT (Generative Pretrained Transformer):

Generative: means it can generate text or other content like image, video etc.

Pretrained: its already trained on massive internet data.

Transformer: the neural network architecture it uses.

Transformer:

Pretraining Phase -----> **Inference Phase**

- Learning phase
- generate response
- Learns through massive dataset (code, book etc.)

- refers

Step1: Input text is broken into tokens.

Input: **Apple a day keeps doctor away**

Tokens: ["Apple", "a", "day", "keeps", "doctor", "away"]

But point to note that token is not equivalent to word.

Input: **Apple makes extraordinary phones**

Tokens: ["Apple", "makes ", "extra", "ordinary", "phones"]

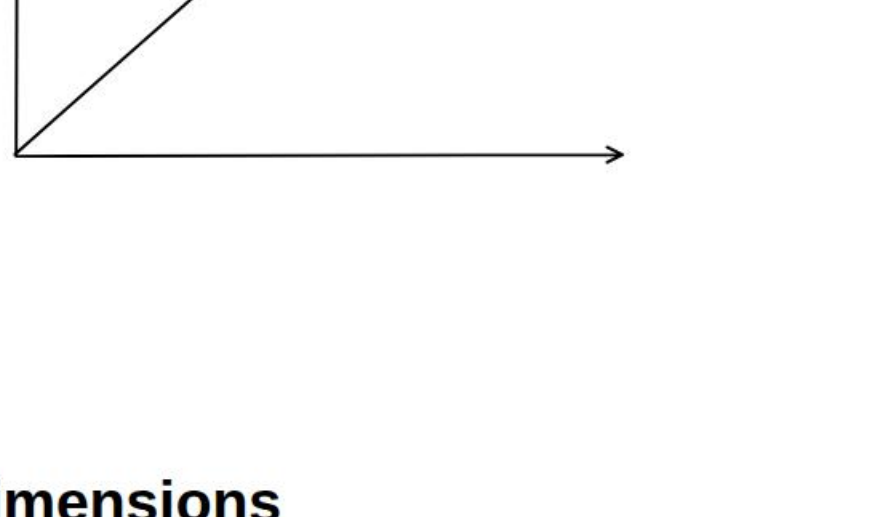
Step2: Compute Static Embedding

Each token is assigned a fixed point in a multi-dimensional space, learned during pretraining from huge amounts of texts.

Tokens: ["Apple", "makes ", "extra", "ordinary", "phones"]

Token	Static Embedding (3-D)
"Apple"	[0.10, 0.45, 0.88]
"makes "	[-0.22, 0.36, -0.11]
"extra"	[0.41, 0.12, -0.33]
"ordinary"	[-0.17, 0.60, 0.09]
"phones"	[0.33, -0.16, 0.40]

3 values in a vector means



GPT-3 : 12,288 dimensions
GPT-4 : haven't published exact size

Step3: Building Contextual Embedding

Now, till now all we have is static vector for a token. And there is no context if you see below :

Static embedding would have generate same vector for token "Apple"

Tokens: ["Apple", "a", "day", "keeps", "doctor", "away"]

Tokens: ["Apple", "makes ", "extra", "ordinary", "phones"]

"Apple" [0.10, 0.45, 0.88]

Even though one Apple refers to fruit and another Company. Both has same static embedding

So, in this step #3, it does "**Self-attention**", a layer which tells how much attention it need to be given to the neighbor token.

Tokens: ["**Apple**", "a", "day", "keeps", "**doctor**", "away"]
could be a fruit, so it will update its vector values to reflect this context.

Tokens: ["**Apple**", "makes ", "extra", "ordinary", "**phones**"]
could be a IT Company, so it will update its vector values to reflect this context

And this vector passes through multiple Self-attention layers. And each layer refine the vector more and more.

So this Step #1, #2 and #3 is called **Encoding layer**.

Now **Decoding Layer** starts, it takes contextual embedding as input and its output is next best possible token.

Token	Contextual Embedding	Next possible token could be:	Token
"Apple"	[0.10, 0.45, 0.88]		"App"
"makes "	[-0.22, 0.36, -0.11]		"ma"
"extra"	[0.41, 0.12, -0.33]	"." -> 0.5	"ext"
"ordinary"	[-0.17, 0.60, 0.09]	"!" -> 0.3	"ord"
"phones"	[0.33, -0.16, 0.40]	"?" -> 0.01	"pho"
			"."

And process repeats, till its response is not fully complete, It also maintains the cache for previously computed contextual embedding vectors

GPT-3.5: has around 96 transformer layer

Now since it based on prediction, how it knows when to stop predicting and respond

GPT has pre-defined EOS (end of sequence) or special tokens like ``.

Now while predicting, next token is predicted is this special token, it stops or

we can also set max token limit in the API, so even if special token is not but token limit is reached, it will stop.

GPT versions:

	Version	Capability
GPT-x	GPT-1 , GPT-2, GPT-3, GPT-3.4, GPT-4, GPT-5	Test focused, code capabilities
GPT-o	GPT-4o	Omni / Multi Modal, supports text, audio, image etc.
GPT-mini	GPT-4o-mini	Mini is small and fast but tradeoff in capabilities
Image Generation Model	GPT-image-1	Generates or edit images
Speech and Text Model	TTS	Text to Speech
	Whisper	Speech to Text
Realtime	GPT-realtime	Optimized for WebSocket and webRTC, do not wait for completion

Spring AI, Project Setup:

Prerequisite: Springboot

2 ways to Integrate our Application with LLM

External Hosted LLM APIs :

We can integrate with external service providers like:

- OpenAI
- Deepseek
- Gemini
- Etc...

Pro:

- Easy setup
- No infra overhead
- Easily Scalable

Cons:

- Cost per usage. So cost increases with usage increase
- Data goes outside your Infra
- Less control over fine tuning

Self Hosted LLM APIs:

Download open source LLM and run on your servers (cloud or locally)

- LLaMA 2/3
- Falcon
- GPT4All
- Etc...

Pro:

- Full control
- Better for privacy
- One time infra cost and no recurring billing

Cons:

- Huge CPU/GPU capability
- Ongoing maintenance
- Difficult to setup

Let's Integrate with External Hosted LLM i.e.

OpenAI:

Step1: Login to OpenAI platform "<https://platform.openai.com/docs/overview>"



Step2: Give Organization name (whatever you like), also in OpenAI, our account can belong to only one organization only

Step3: Give the name of the 1st Project (whatever we like) and also provide the name for the Secret Key

API key name

MySecretKey

Project name

LearnGenerativeAI

API keys belong to projects to help you manage usage limits, team access, and data security.

Generate API Key

Step4: Copy the **Secret Key** (and never share it with anyone). Because using this only OpenAI does authentication and allow access to its APIs.

we can also manage the keys, says we can disable or remove the old key and create new one.

Step5: Purchase some credit, as we are integrating with External Service provider, there is a charge for every request.

If we don't have the sufficient credit, we won't be able to use their APIs.

☒ \$5 credits Recommended

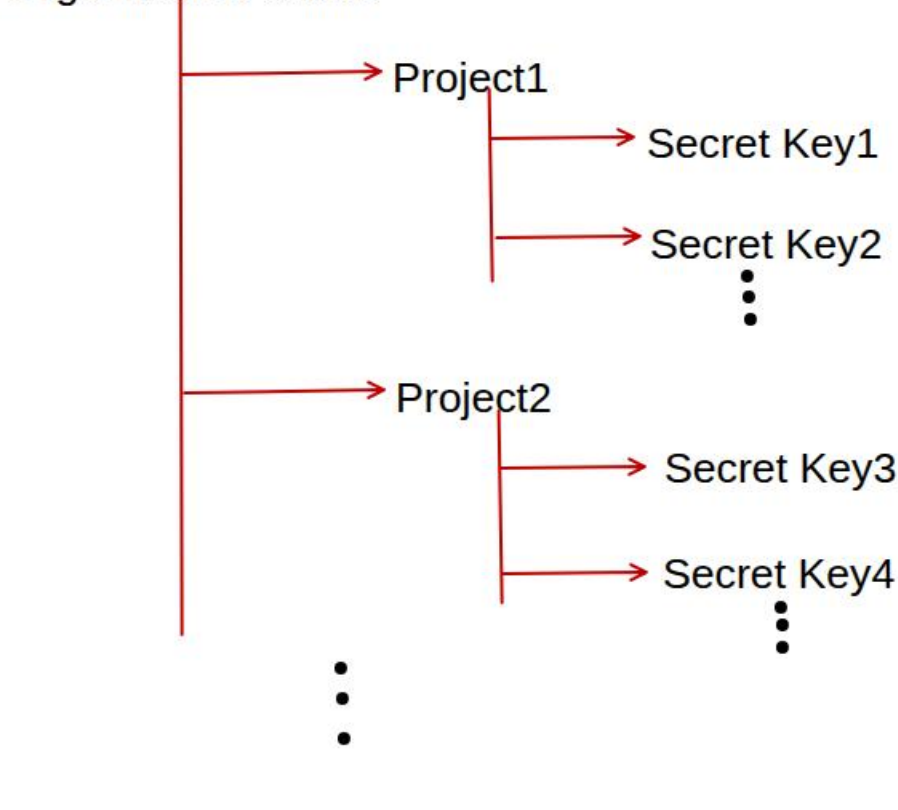
☐ \$10 credits

☐ \$20 credits

\$5 can cover about 2 million input or 500k output tokens. Plenty to start building your idea. [View pricing](#)

Purchase credits

- At org level, we can set the limit, like how many requests are allowed.
- Rate limiting is also at org level only
- Project level, we can only see the API keys



Step6: Create a Springboot Application

<https://start.spring.io/>

Adds this dependency

```
3 #never add your secret key in application.property
4 # either set it in environment or
5 # store it in key management service on server
6 spring.ai.openai.api-key=${OPENAI_API_KEY}
7
```

Setting my Secret Key in Environment variable:

Edit configuration templates...

?

Run Cancel Apply OK

Edit configuration templates...

?

Run Cancel Apply OK

We have already done Project setup in previous topic:

Today lets see, how we can use Spring AI APIs to communicate with these AI providers

Start the server:

How it worked?

In the above code, I did not mention which AI provider (OpenAI or Anthropic etc) sending the request to?

What API of the API provider do I need to invoke?

so how its working?

That's Spring AI framework, it hides all the complexity from us.

BUT... lets dig further and see how its working internally.

So, lets see the code again.

Lets deep dive and see how it might be actually working:

Message:

Message represent one message (single conversation) in a chat between user and AI.

Message Types:

- System Message: defines AI's behavior like you are a Java tutor, you are a customer support agent etc.
- User Message: this represents User input or query, which it want to ask to AI.
- Assistant Message: AI's previous response, used to recall and maintain the context.

Message msg = new SystemMessage("You are a Java tutor");
msg.getMessageType(); // SYSTEM
msg.getContent(); // You are a Java tutor

ChatOptions:

Using this, we can control the runtime behaviour of AI model.

- Control Creativity level - number lies between 0 and 1.
 - 0 : more deterministic. (very predictable, as it picks highest probability token)
 - 1 : output can be more creative in sense say, more diverse, unpredictable
- Control the Length of the response like using 100 words only.
- Control the AI model to use for the message like use GPT4 or use GPT3 etc.
- Control the number of responses - say I want 2 different response for a question and then I can use N(2) whatever version I like.

ChatOptions options = OpenAIChatOptions.builder()
.model("gpt-4o-mini")
.temperature(0.5)
.frequencyPenalty(1.0)
.maxTokens(300)
.maxCompletionTokens(100)
.N(1)
.stop(List.of("END"))
.build();

model	Which model to use for this conversation
temperature	Number between 0.0 and 1.0, say 0.2 more deterministic and predictable while 0.8 creative, randomness
frequencyPenalty	Number between -2.0 and 2.0, positive number means more penalty, means less probability of repeating the token. example: Sky is this, sky is that, sky is everywhere. but with penalty say, (1.5): Sky is that, sky is that, its everywhere.
maxTokens	Limit the output length, means how much max token is allowed to be consumed for AI generation. Used with Non Reasoning Model like gpt-4o-mini, gpt-3.5-turbo These model good for chat, content generation etc.
maxCompletionTokens	Used with Reasoning Model like gpt-4o-reasoning, gpt-4-turbo-instruct These model good for Math's problem, puzzle etc. say: maxCompletionTokens = 300 then internally it can decide: 200 token use for doing reasoning (thinking for better answer, which require extra tokens) and 100 token use for producing output
N	Number of output choices you want. N=2 means for each query, AI will respond with 2 version of the output.
stop	Defines one or more stop Sequence, where the AI predict one of these tokens, stop the generation.

Prompt: this represents the input that is sent to the AI model.

List<Message> messages = List.of(
new SystemMessage("You are a tutor."),
new UserMessage("Please Explain with example"),
new AssistantMessage("Polymorphism lets one object take many forms depending on the runtime")
);
ChatOptions options = OpenAIChatOptions.builder()
.model("gpt-4o-mini")
.temperature(0.5)
.frequencyPenalty(0.5)
.maxTokens(200)
.stop(List.of("END"))
.build();
Prompt prompt = Prompt.builder()
.messages(messages)
.chatOptions(options)
.build();

ChatModel:

Now say, we have created the Prompt, but this prompt is generic and agnostic of AI provider. We can not send directly this Prompt object to AI provider.

Say when we have to invoke OpenAI, this Prompt need to be converted in a format which AI understands. Similarly for other Providers, prompt need to be formatted accordingly.

Methods in ChatModel	Usage	Example
String call(String m)	Default method present in interface. 1. creates prompt object 2. And then pass it to specific AI provider ChatModel Implementation	Prompt prompt = new Prompt(new UserMessage(m)); ChatResponse response = call(prompt); //each specific AI provider has its own call(prompt) method
String call(Message... m)	Default method present in interface. 1. creates prompt object with multiple messages. 2. And then pass it to specific AI provider ChatModel Implementation	Prompt prompt = new Prompt(Arrays.asList(m)); ChatResponse response = call(prompt); //each specific AI provider has its own call(prompt) method
ChatResponse call(Prompt prompt)	Each specific ChatModel provides an implementation and internally invokes the AI provider API.	@Override ChatResponse call(Prompt prompt); implementation by OpenAIChatModel or AnthropicAIChatModel
ChatOptions getDefaultOptions()	Create default ChatOptions object, if we don't want to manually create one.	

Let's see one full example:

What if you have to use more than 2 AI Providers?

Then we have to specifically mentioned the ChatModel object, then Auto configure will not work.

Using Anthropic provider:

Using OpenAI provider:

ChatResponse and Generation:

Generation: Response Generated by AI

Contains List<Generation> + some additional information

ChatResponse:

Prompt prompt = new Prompt(
List.of(
new SystemMessage("You are a tutor."),
new UserMessage("Explain polymorphism in short")
),
OpenAIChatOptions.builder()
.model("gpt-4o-mini")
.temperature(0.5)
.N(2) // ask for 2 completions
.build()
);
ChatResponse response = chatModel.call(prompt);

What the AI returns conceptually, each AI provider response fields could be different.

Mapped to ChatResponse

ChatResponse make our code agnostic of different AI provider response.

Generation generation = response.getResult();

Generation	getResult();	Returns the first in the generation list	Generation gen = response.getResult();
List<Generation>	getResults();	Returns list of generated output	List<Generation> genList = response.getResults();

AssistantMessage
getOutput();

output from AI provider:
AssistantMessage aiResponse = generation.getOutput();
aiResponse.getText(); //actual text message from AI
aiResponse.getMessageType(); //User or System or Assistant, in response Assistant only, as it's a response from AI

Structured output:

ResponseFormat: Using this object, we can specify the format in which AI must output.

JSON_OBJECT

This guarantees that, the message the model generates is valid JSON form.

Need to provide 'json' word in message too.

Generates a text response and it's a default behavior.

JSON_SCHEMA:

